

# AWS Cloud Foundations

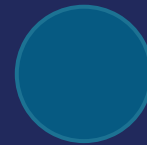
*Compute • Storage • Identity*



COMPUTE



STORAGE



IDENTITY

# AWS Compute Essentials

## WHAT'S AHEAD

- Why the cloud, and the AWS Shared Responsibility Model
- AWS's service landscape — where EC2, S3, and IAM fit
- AWS global infrastructure: Regions & Availability Zones
- Compute & networking essentials: EC2, VPC, Security Groups
- Basic Auto Scaling groups

# Why Not Just Buy Hardware?

- Traditional infrastructure asks a team to commit to hardware before it knows what it actually needs
- Real procurement and setup takes real time — a project stalls waiting for servers to arrive and be configured
- To survive an unpredictable traffic spike, a team over-provisions for its worst day
- Most of that capacity sits idle the rest of the time; reaching a new geography means building infrastructure there too

PROBLEM

## TRADITIONAL IT

Buy hardware → wait weeks/months → hope you sized it right

## MOSTLY IDLE CAPACITY

Sized for the one day a year you needed it — idle every other day

*A data center rack, a slow procurement clock, and mostly-idle capacity*

# Pay-as-You-Go Cloud Infrastructure

SOLUTION

- Cloud model: pay-as-you-go, access compute, storage, and networking on an as-needed basis
- Provision in minutes instead of months
- Scale up or down instantly as real demand changes
- Reach global users without ever building a physical data center

## CLOUD

Provision in minutes → pay for what you use → scale instantly

## GLOBAL REACH

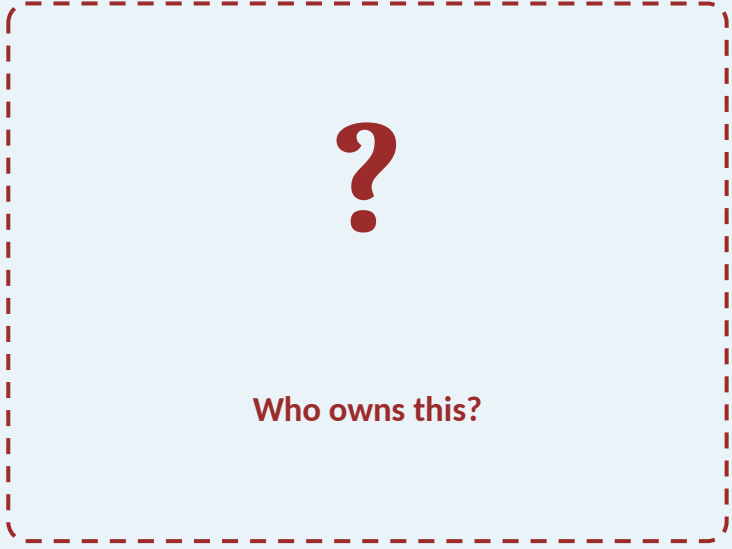
Reach users worldwide without building a single data center

*A dial that scales instantly, and reach without new data centers*

# Who's Responsible for Security Now?

PROBLEM

- Moving infrastructure to AWS means AWS now runs the physical hardware, not your team
- Security doesn't just disappear as a concern because someone else owns the servers
- A team needs to know exactly what it's still on the hook for, and what it isn't
- Without a clear line, a real gap (an open port, a public bucket) gets missed until it's exploited



Who owns this?

*A question mark over AWS's infrastructure — who secures what?*

# The AWS Shared Responsibility Model

SOLUTION

- AWS secures the cloud: physical facilities, hardware, networking, hypervisor, host OS, Regions/AZs
- Customer secures what's in the cloud: guest OS & patches, security group rules, data & encryption, IAM
- The split shifts by service — EC2 leaves more to the customer than a managed service like S3
- Today's defaults (security groups deny inbound, S3 blocks public access) are the customer's half of this line

AWS

Security OF the Cloud  
hardware · hypervisor · regions

CUSTOMER

Security IN the Cloud  
guest OS · security groups · data · IAM

*AWS's half vs. the customer's half of the security line*

# AWS at a Glance

- AWS groups services into categories: Compute, Storage, Databases, Networking & Content Delivery, Machine Learning/AI
- Security & Identity (where IAM lives) cuts across every category
- Today's foundation: EC2 (Compute), VPC (Networking), S3 (Storage), IAM (Security & Identity)

## Compute

EC2 • Lambda

## Storage

S3 • EBS

## Databases

RDS • DynamoDB

## Networking

VPC • Route 53 • CloudFront

## ML / AI

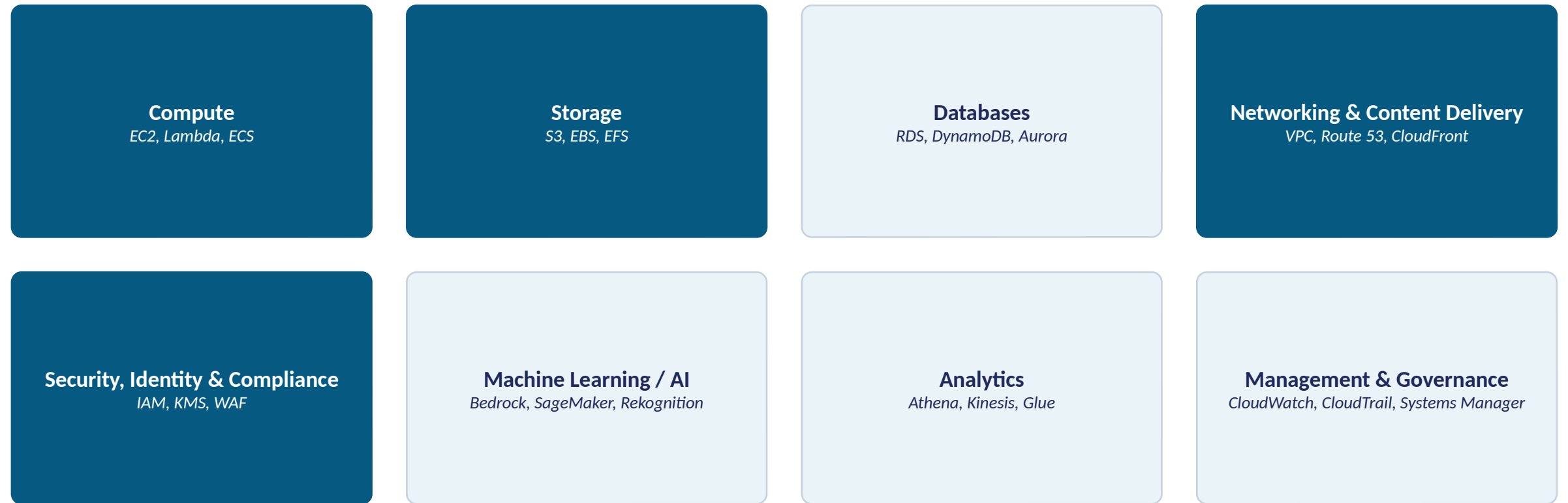
Bedrock • SageMaker

*IAM: Security & Identity, spans every category above*

*AWS service categories, with EC2/VPC/S3/IAM highlighted*

# The Wider AWS Landscape

Where EC2, VPC, S3, and IAM sit among AWS's most common services, by category



*Highlighted: where this week's work lives (Compute/EC2, Networking/VPC, Storage/S3, Security/IAM)*



# Why Put All Your Eggs in One Data Center?

PROBLEM

- An application has to run somewhere physically real: one building, one power feed, one network link
- If that single "somewhere" fails, a power outage, a fire, a cut fiber line, the whole application goes down
- No amount of good code protects against losing the one physical place it all runs
- Before touching the console, the team needs a mental model for where anything they build actually runs

1 DATA CENTER

1 power feed • 1 network link

Whole application: DOWN

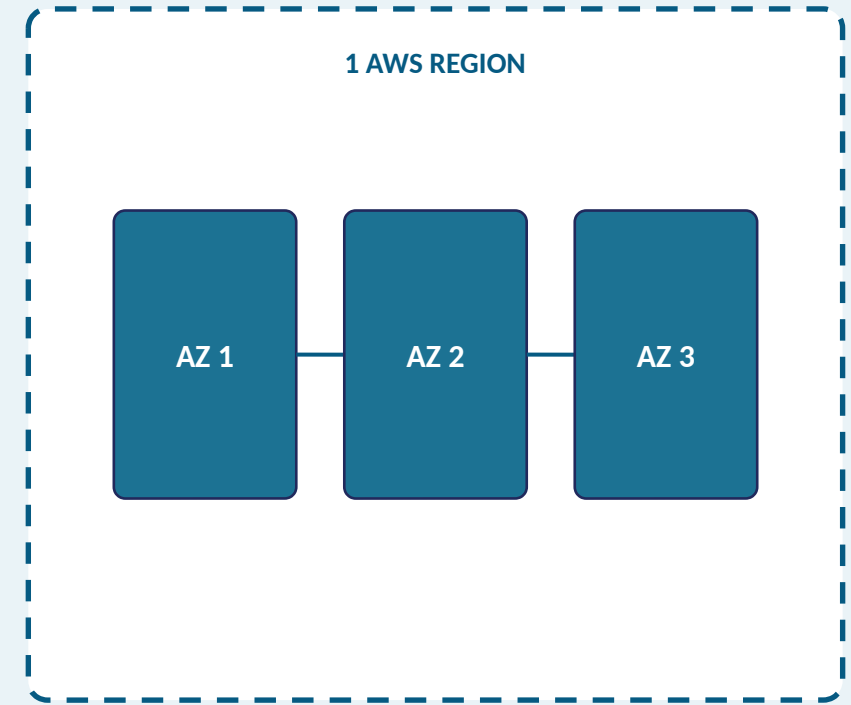
*nothing else to fail over to*

*One data center, one bad day, the whole application down*

# AWS Regions & Availability Zones

SOLUTION

- Region: a physical location where AWS clusters data centers (39 regions today)
- Every region has 3+ Availability Zones (AZs) — 124 AZs total
- An AZ = one or more data centers with its own power, networking, connectivity
- Far enough apart to survive an outage, close enough for low-latency links
- Your team's Region choice today is sticky — pick one and keep it all week

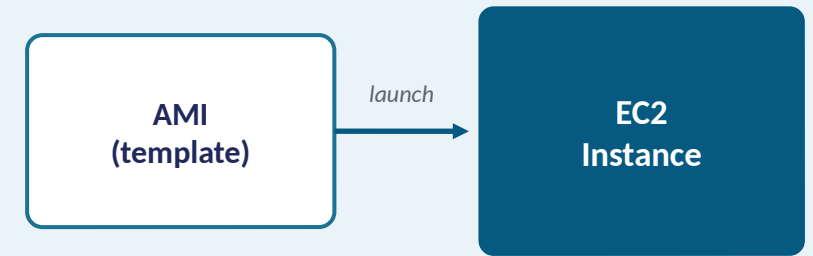


*One region containing three Availability Zones*

# EC2 Instances & AMIs

- EC2 = Amazon's virtual server service
- Instance = one running virtual machine
- AMI (Amazon Machine Image) = the template an instance launches from
- Instance type sets vCPU, memory, network, and cost — resizable later
- Key pair = the credential used to connect over SSH

**HANDS-ON** Launch an EC2 instance from an AMI

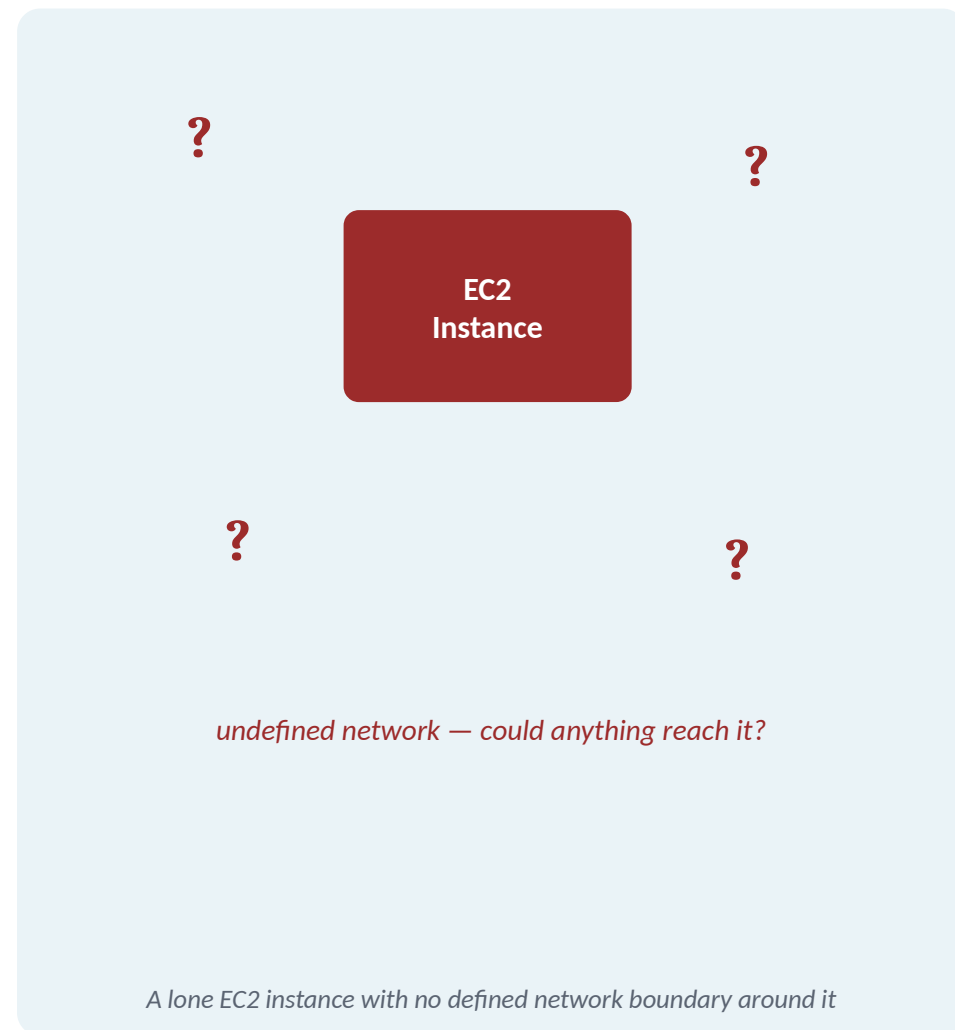


*Launching an EC2 instance from an AMI template*

# Whose Network Is My Instance On?

PROBLEM

- Every EC2 instance has to live somewhere on a network the moment it launches
- A wide-open, shared, or unmanaged network means anything could potentially reach it
- Before a security group can filter traffic, something has to define the network itself
- That question — which network, which IP range, which zone — has to be answered first

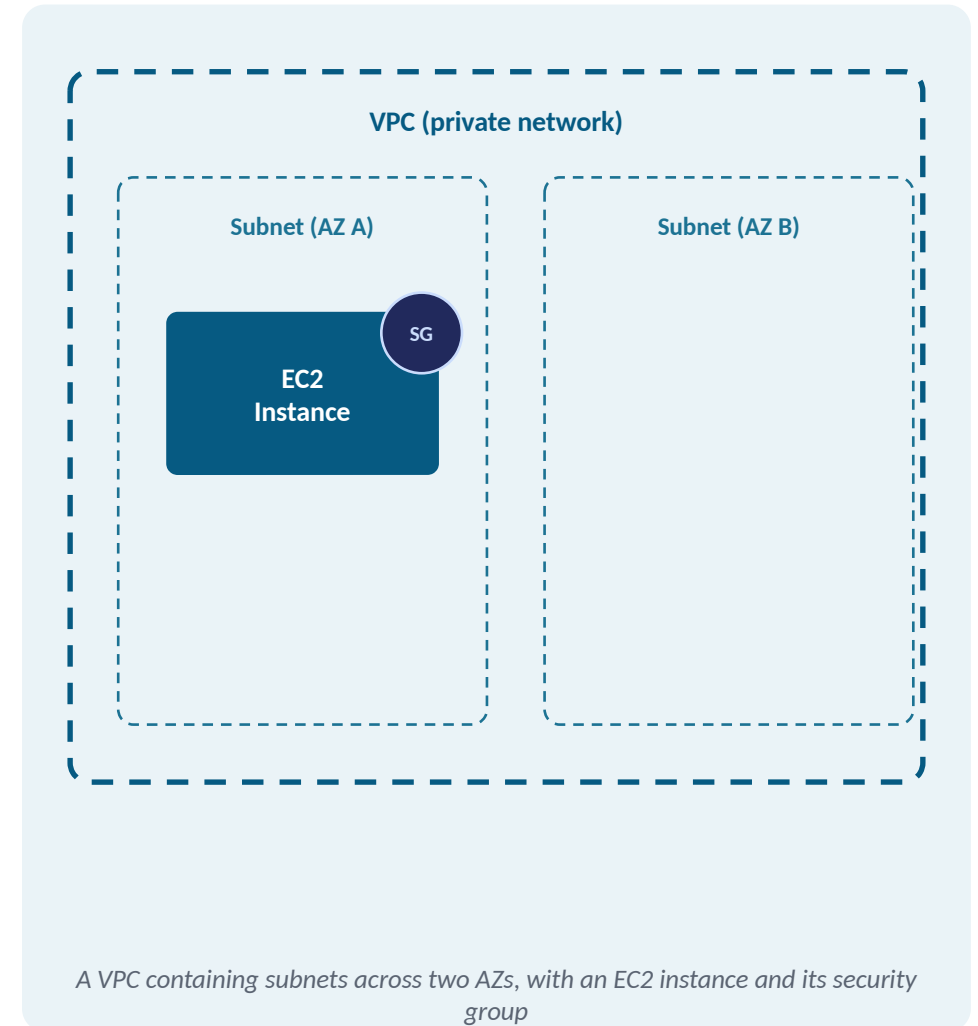


# Amazon VPC: Your Private Network

SOLUTION

- A VPC is your own logically isolated private network inside AWS
- Subnets subdivide a VPC; each subnet lives in exactly one Availability Zone
- Every account gets a default VPC per Region, pre-configured so instances launch immediately
- Security groups (next) filter traffic on top of whichever VPC and subnet an instance is in

HANDS-ON Part of: Launch an EC2 instance from an AMI



# Security Groups: Your Instance's Firewall

- A security group is a stateful virtual firewall for your instance
- Default: no inbound traffic allowed, all outbound traffic allowed
- An inbound rule = Type + Port range + Source
- AWS guidance: never open SSH/RDP to 0.0.0.0/0



your IP, app port → allowed

0.0.0.0/0 → blocked by policy

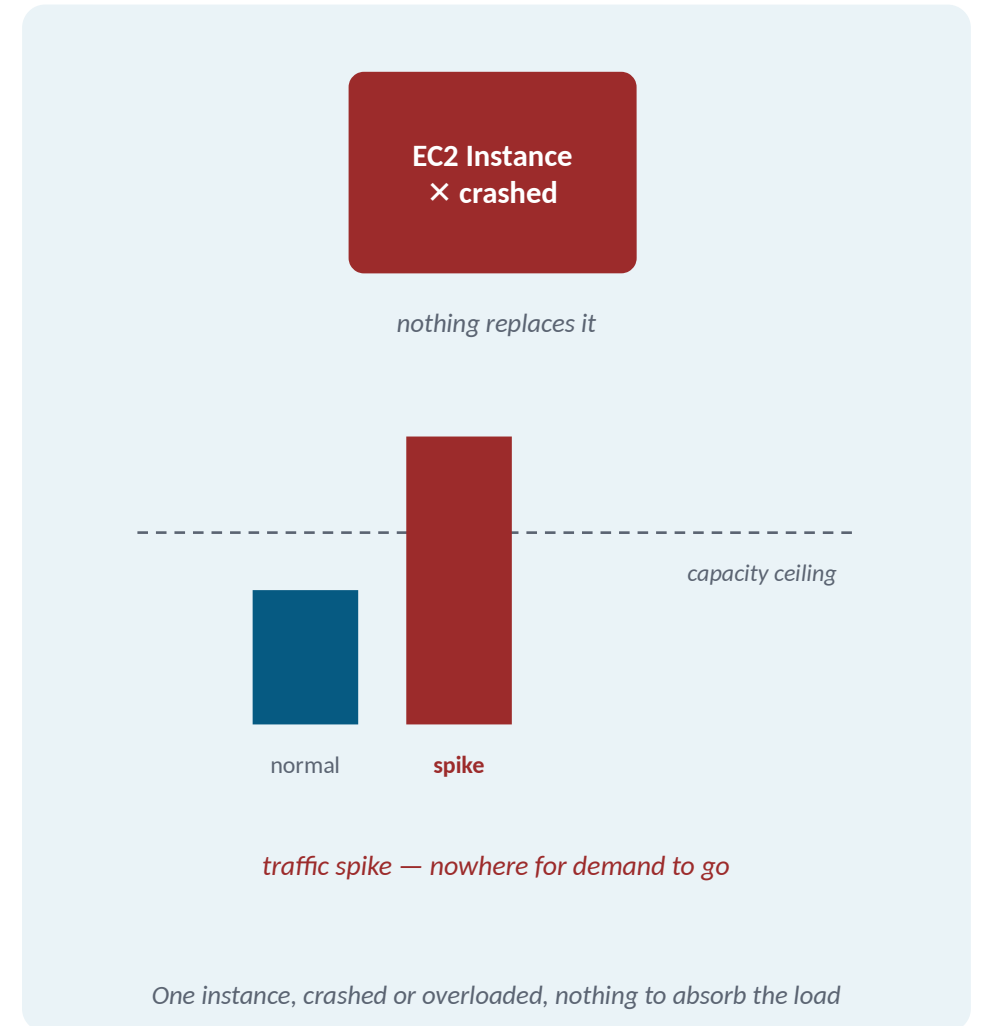
*Security group filtering inbound traffic to EC2*

**HANDS-ON** Open the port the capstone app needs

# What Happens When One Instance Isn't Enough?

PROBLEM

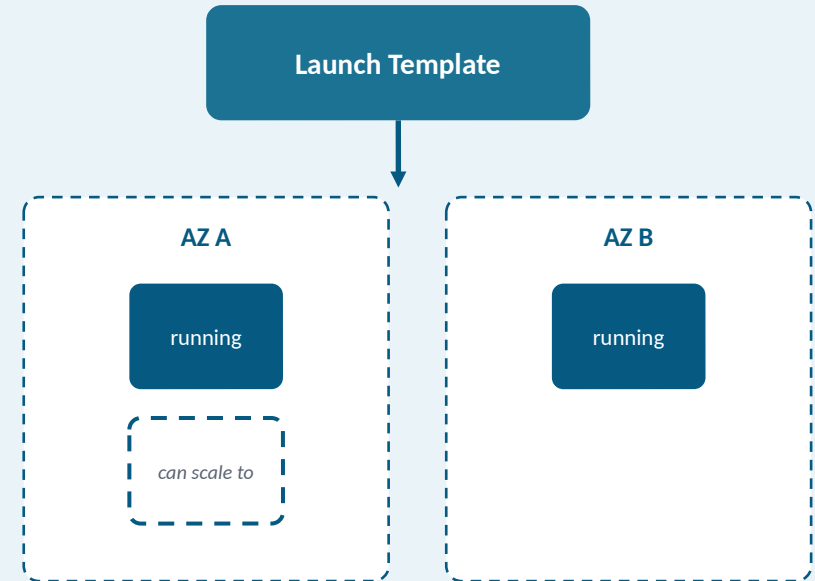
- A single running instance is itself a single point of failure: if it crashes, nothing replaces it
- No self-healing: the application stays down until someone manually launches a new instance
- No automatic response to a traffic spike: extra demand has nowhere to go
- Scaling back down after a spike means someone watching a dashboard and terminating instances by hand



# Auto Scaling Groups: Desired, Min, Max

SOLUTION

- Launch template = saved, reusable instance definition (AMI, type, key pair, security groups)
- Desired capacity: how many instances should run right now
- Minimum / Maximum capacity: the floor and ceiling the group can scale between
- Spreading instances across 2+ AZs makes the fleet resilient



Desired: 1 Min: 1 Max: 2

*One launch template feeding an Auto Scaling group*

HANDS-ON Configure a basic Auto Scaling group



# AWS Storage & Identity Essentials

## WHAT'S AHEAD

- AWS's storage landscape: S3, EBS & EFS
- S3 buckets, storage classes & lifecycle policies
- IAM: users, roles, policies & permissions, least-privilege design
- Instance profiles: attaching a role to EC2
- Completing the capstone's AWS foundation

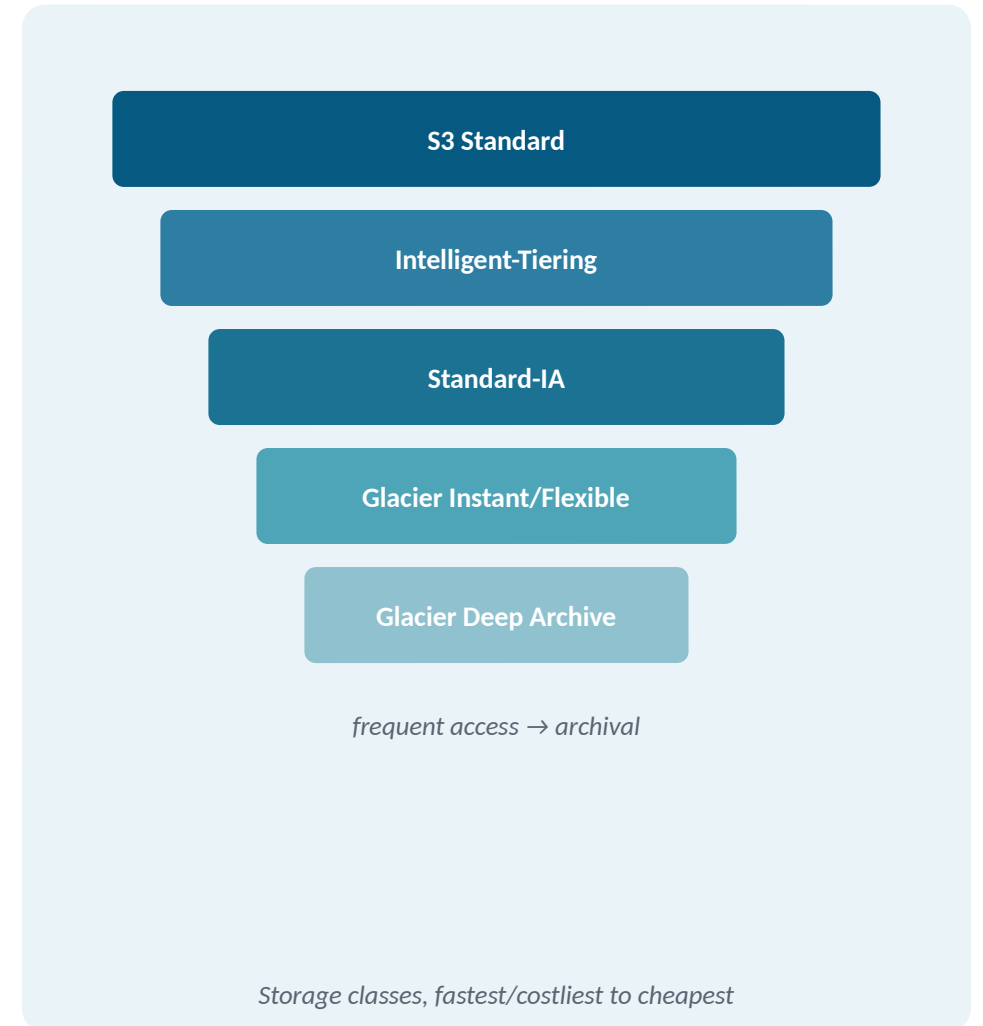
# AWS's Storage Landscape

- S3 (object storage): unlimited unstructured data, accessed by API, not attached to any one instance
- EBS (block storage): behaves like a hard drive, attached to exactly one EC2 instance at a time
- EFS (file storage): a shared filesystem multiple instances can mount and use at the same time
- Today's course goes deep on one of these, S3, because it's what the capstone application needs



# S3 Buckets & Storage Classes

- S3 stores data as objects inside buckets — bucket names are globally unique
- A bucket's Region can't be changed after creation
- Keep the defaults: Bucket owner enforced, Block Public Access on, SSE-S3 encryption
- Storage classes trade cost vs. retrieval speed



# What Happens to Objects Nobody Downgrades?

PROBLEM

- Left alone, every object stays in S3 Standard, at Standard's price, forever
- Whether an object is read every second or never touched again, the bill doesn't know the difference
- Managing this by hand means someone has to remember, every month, to find and downgrade old objects
- That reliably stops happening once the person is busy, on leave, or moves to a different team



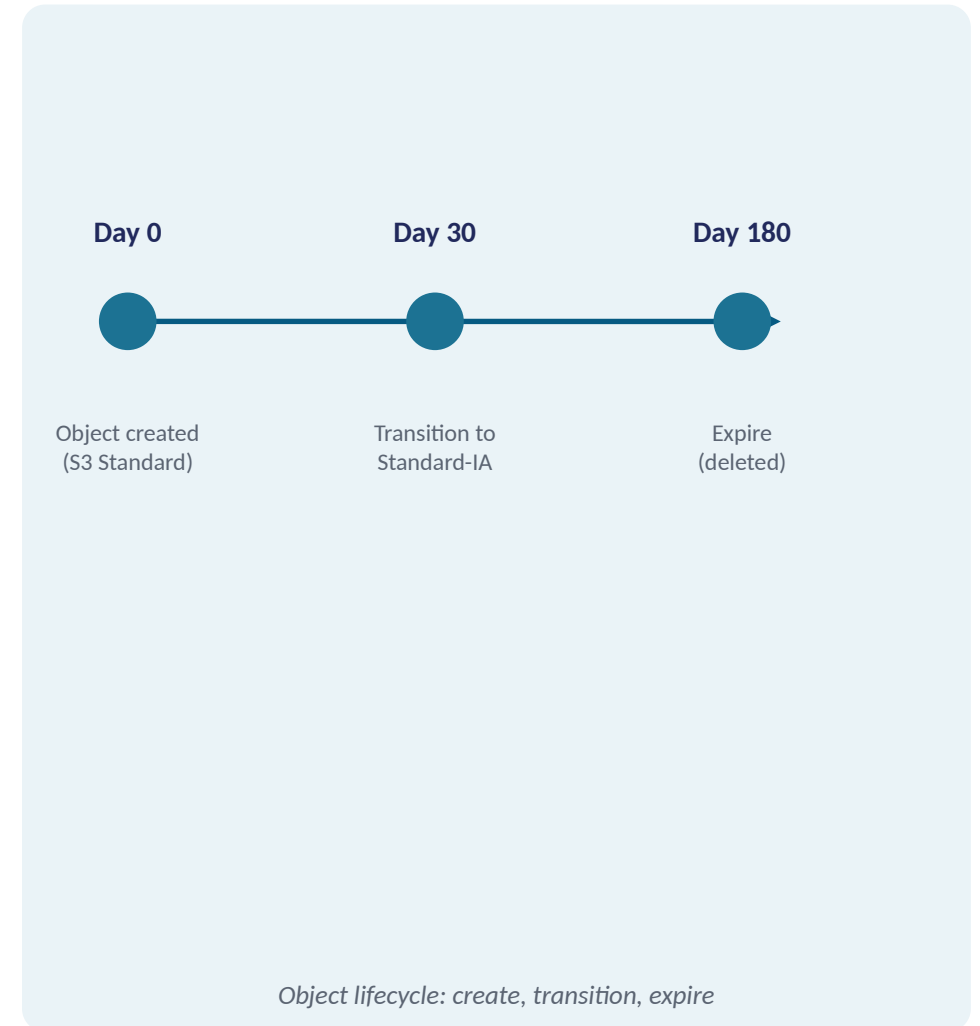
*An ever-growing bucket and a rising bill nobody is watching*

# S3 Lifecycle Policies

SOLUTION

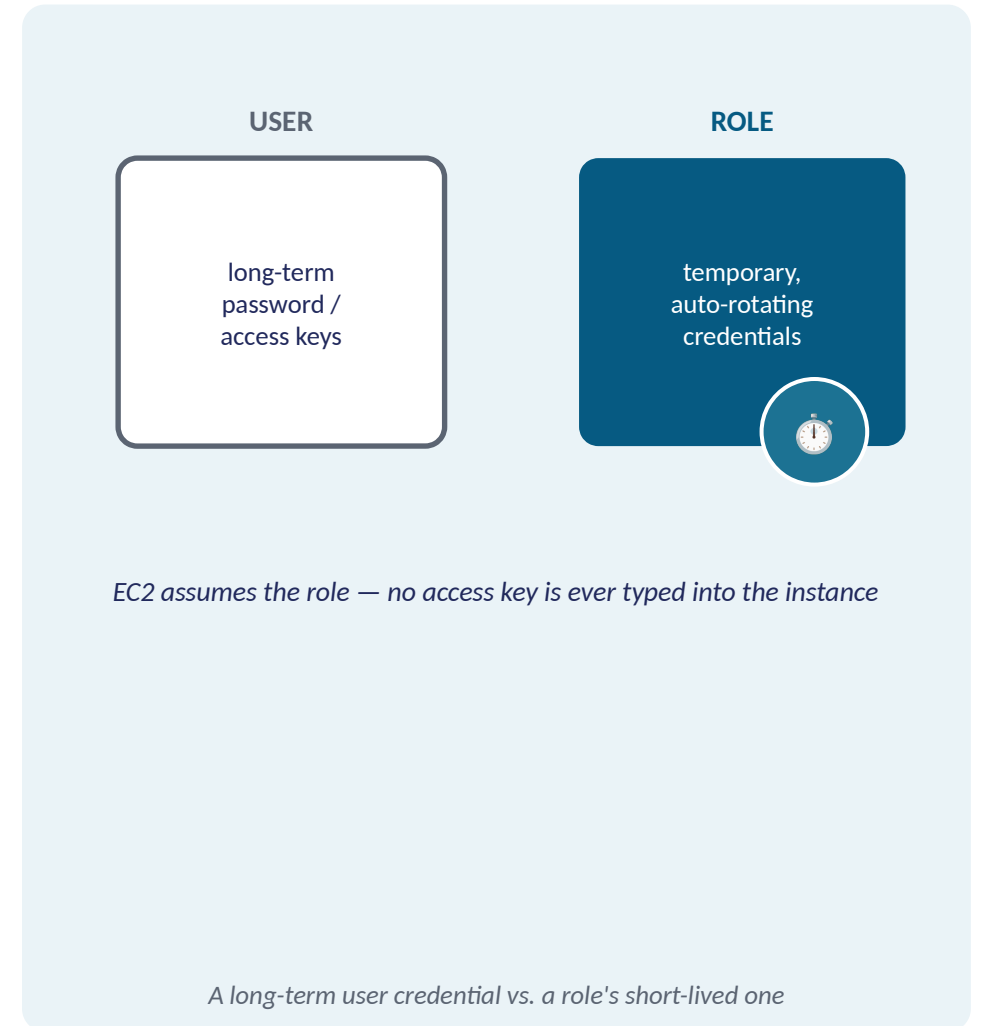
- A lifecycle rule automates moving or deleting objects over time
- Scope: all objects in the bucket, or a specific prefix/tag
- Transition action: move to a cheaper storage class after N days
- Expire action: delete the object after N days

HANDS-ON Create an S3 bucket + lifecycle policy



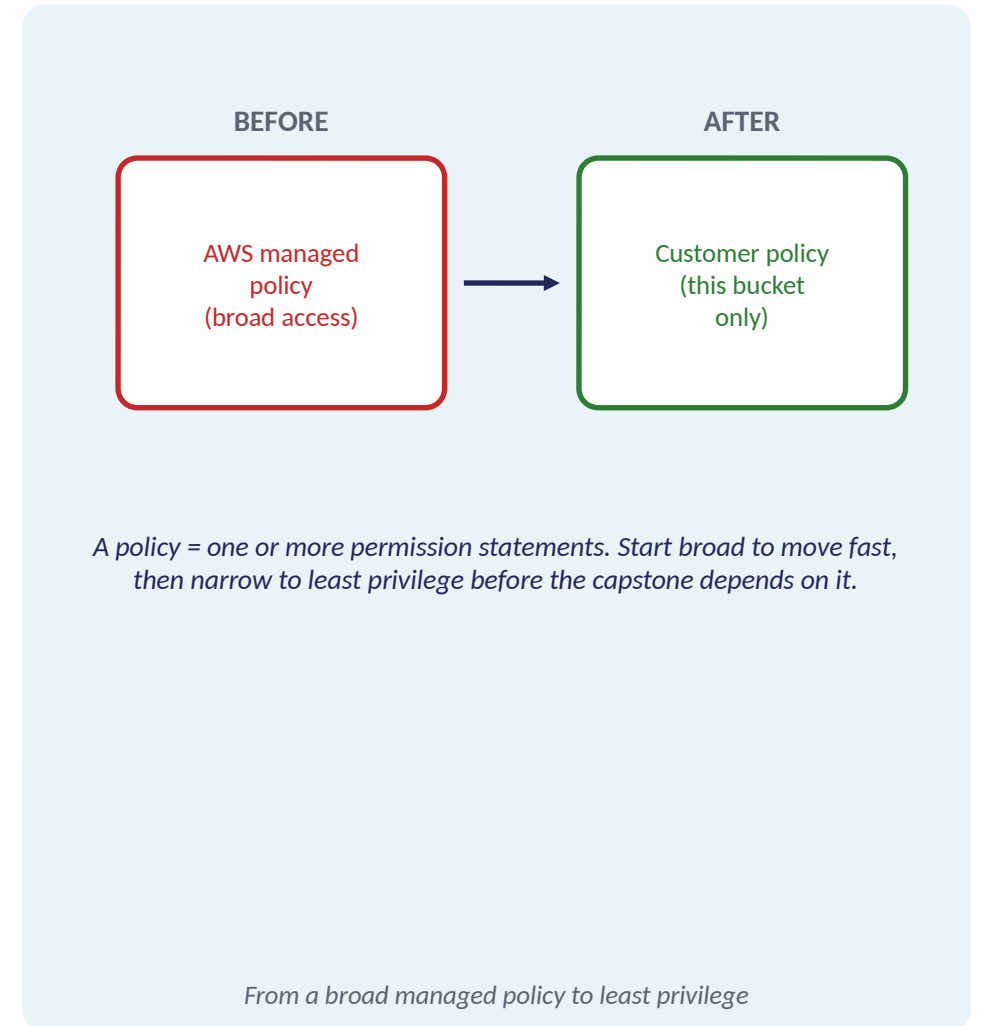
# IAM: Users vs. Roles

- User: an identity for one person or app, with its own long-term password and/or access keys
- Role: an identity with no long-term credentials, assumed temporarily for one session
- Assuming a role issues short-term, automatically-rotating credentials, not a standing secret to leak
- AWS's guidance: workloads on AWS compute (like EC2) should use roles, not a copied user access key



# IAM: Policies, Permissions & Least Privilege

- Permission: one statement, this action, on this resource, allowed or denied
- Policy: a JSON document made of one or more permission statements
- A policy attaches to a user, a role, or a group, and can be reused across many identities
- Least privilege: start with a managed policy to move fast, then narrow to name only what's needed



# Instance Profiles: Attaching a Role to EC2

- Instance profile = the container IAM builds around a role for EC2
- Only one role per instance at a time (reusable across many instances)
- Console path: Actions → Security → Modify IAM role
- Together: compute + storage + identity = the capstone's AWS foundation

**HANDS-ON** Attach the role; provision the AWS foundation

